

# RoTE 在 ReChorus 框架中的 CPU 复现与序列推荐对比

25 组 史熠鑫 (24350124) 王宸 (23330123)

## 摘要

序列推荐依据用户过去的交互预测下一件可能感兴趣的物品。普通位置编码主要描述先后顺序，难以直接表达相邻行为间的真实时间间隔。本文依据 SIGIR 2026 的 RoTE 论文，在 ReChorus 框架内实现按年、月、日三个层级旋转注意力查询和键的模型，并在 Grocery\_and\_Gourmet\_Food、MovieLens-1M 两套数据上与框架自带的 SASRec、GRU4Rec 比较。实验仅使用 CPU，统一模型规模、训练预算、候选集和三组随机种子，报告 HR@5/10、NDCG@5/10 的均值与标准差，并公开数据准备、审计和运行脚本。

## 1 研究背景与任务

推荐系统常将用户行为按时间排序，再利用最近若干交互推测下一次选择。其困难在于同样的序列位置差可能对应完全不同的实际时间跨度：两次点击可以相隔几分钟，也可以相隔数月。只使用位置编号时，模型无法直接区分这两种情形。传统序列方法 GRU4Rec 使用循环神经网络压缩行为历史；SASRec 通过因果自注意力寻找对下一次行为有帮助的历史项，但标准位置嵌入仍以序列位置为基础。RoTE 提出将原始时间戳拆成粗到细的年、月、日累计序号，再将这些信息放入注意力的查询与键，使相关性计算随时间差变化。

课程指定 ReChorus 作为复现框架。该框架把数据读取器、推荐模型和训练器分开，允许新算法沿用已有的数据接口、训练流程和评价实现。本实验选择同属序列推荐类别的 SASRec 与 GRU4Rec 作为至少两个对照，在框架原有 Grocery 数据和框架列出的 MovieLens-1M 数据上运行。实验关注三个问题：所实现模型能否在 CPU 环境中完整训练和评价；时间旋转是否在固定候选集下带来收益；性能差异是否受数据时间结构、模型容量和训练预算影响。

## 2 技术原理、创新点与具体实现

记一名用户的已观察行为为  $S=\{(i_1, t_1), \dots, (i_n, t_n)\}$ ，目标是给下一件候选物品打分。时间  $t$  为 Unix 秒。参考论文定义，先以 UTC 历法转换为从 1970 年 1 月 1 日起累计的年、月、日序号  $y, m, d$ 。例如 1970 年元旦为  $(0, 0, 0)$ ，1971 年元旦为  $(1, 12, 365)$ 。这里的“月”和“日”是累计值，不是日历中的月份 1 至 12、日期 1 至 31；否则跨年后的距离信息会丢失。

在每个注意力头内，模型对物品嵌入投影得到  $Q, K, V$ 。对  $Q, K$  的相邻偶奇维组成二维向量，以每个时间层级各自的频率进行旋转。频率按 `head_dim` 中偶数维位置的指数衰减，三个层级的频率基数分别为  $10^6, 10^4, 10^2$ 。得到的三个查询向量按 1.5、1.0、0.5 加权求和；键向量使用同样方式。随后对融合后的  $Q$  和  $K$  做缩放点积注意力。 $V$  不旋转，后续残差、归一化与前馈网络仍承担序列表示学习。候选得分为最终历史状态与候选物品向量的内积。训练目标和候选排序由 ReChorus 的 `SequentialModel`、`SeqReader`、`BaseRunner` 提供。旋转没有额外可训练参数，但融合改变了注意力数值，训练仍需重新完成。

论文的创新点在于用累计年、月、日三种时间尺度旋转  $Q$  与  $K$ ，再融合各尺度的注意力表示。一次行为的日期因而进入相关性计算：两条行为即使在序列中相邻，也可以因实际发生时间不同而得到不同的注意力分数。

这里复现的是论文提出的方法；本作业没有提出新的时间编码算法。作业中的工作是把这一模块接入 ReChorus 的序列接口，并在相同候选集和训练预算下检验它与框架算法的差异。

实现文件 `src/models/sequential/RoTE.py` 允许通过 `--model_name RoTE` 调用。模型读取 `history_items`、`history_times` 和序列长度，保留框架对候选物品的打分接口。因果掩码阻止当前位置看到后续行为；历史填充项不会成为有效键。日期变换在 CPU 端完成，之后送入模型张量。该实现以框架的序列训练协议为主，并不宣称逐行复刻作者仓库的训练配置。与框架 SASRec 相比，RoTE 去掉位置嵌入并重写注意力模块；所以结果反映的是这一整套实现的差异，不能完全归因于单个时间旋转操作。

### 3 数据准备与审计

MovieLens-1M 的原始文件给出用户、电影、评分与时间戳。我们仅保留评分至少 4 的正反馈，反复删除交互数少于 5 的用户与物品，直到满足 5-core 条件。随后按每名用户的时间及原文件行号稳定排序，以最后第二次交互验证、最后一次作测试；如留出物品在训练中从未出现，则移除相应用户并重新检查。用户与物品重新映射为从 1 开始的连续整数。最终为 6034 名用户、3125 件物品，训练 562308 条、验证 6034 条、测试 6034 条。这是本实验从框架列出的原始 MovieLens-1M 构造的新划分，不是框架笔记本的原始 Top-K 划分。

Grocery\_and\_Gourmet\_Food 直接来自固定版本的 ReChorus。原数据的验证集有 6336 行、测试集有 6363 行包含重复负样本，会使实际候选集不等价。为保证一条目标与 99 个不同负样本，对两套数据均用固定种子 2026 为每条验证和测试记录从该用户未交互物品中无放回抽取 99 件。Grocery 保持全部训练、验证、测试正样本及时间不变，只替换负候选；派生数据为 14681 名用户、8713 件物品，训练 121892 条、验证和测试各 14681 条。审计发现 Grocery 有 26 件留出正物品未在训练中出现，属于冷启动物品，三种模型会面对同一问题，解读 Grocery 的绝对分数时须注意这一限制。

`audit_data.py` 核验用户和物品 ID 范围、时间顺序、每行负样本数及重复、负样本是否与该用户任一交互物品冲突。两套派生数据均未检出时间顺序错误、重复负样本、已交互负样本或越界候选；MovieLens 留出物品全部出现在训练中。划分与负样本文件固定并共享，不随模型或训练随机种子变化。

### 4 实验设置与验证

所有实验在 Windows 的 Python 3.10.11、PyTorch 2.2.2 环境使用 CPU。每个模型在每套数据上分别取随机种子 0、1、2，共 18 次正式运行。统一历史截断长度 20、嵌入维度 16、批量 256、学习率 0.001、最多训练 8 轮，验证集 NDCG@10 选最佳轮次，早停耐心值为 3。RoTE 和 SASRec 均使用 1 层、2 个注意力头，GRU4Rec 隐层为 16。模型容量及运行时间分别由实际日志记录，不以相同维度假定参数量完全一致。

测试阶段为每条目标在一个正物品与 99 个固定负物品中排序，计算命中率 HR@5、HR@10 以及折损累计增益 NDCG@5、NDCG@10。每项指标对三次运行报告算术均值与样本标准差；标准差用于显示随机初始化带来的波动，而不是显著性证明。训练耗时包含完整运行进程的时间，因此也受数据读取和 CPU 工作负载影响。MovieLens 的多个模型同时运行，时间记录存在资源竞争，不用于效率排名。原论文使用其他 Amazon 数据、最长历史 50、五个随机种子及不同训练和评价设置，两者的绝对数值不宜直接比较。

正式训练前，自动测试覆盖 UTC 日历分解、二维旋转范数不变、时间变化对输出的影响和 CPU 反向传播，并用小数据跑通三种模型的 Reader→Model→Runner 路径。随后对每个数据集与模型组合进行单轮检查。数据准备脚本、环境依赖、固定参数运行器、全部命令与原始日志随代码提供；按照 README 可从原始数据重新生成派生文件与汇总表。

## 5 结果与讨论

表 1 给出 Grocery 测试集的三种子均值  $\pm$  样本标准差，所有指标均在每条记录的 100 个固定候选中计算。

| 模型      | HR@5                | NDCG@5              | HR@10               | NDCG@10             |
|---------|---------------------|---------------------|---------------------|---------------------|
| RoTE    | 0.3498 $\pm$ 0.0070 | 0.2445 $\pm$ 0.0044 | 0.4551 $\pm$ 0.0104 | 0.2786 $\pm$ 0.0044 |
| SASRec  | 0.3469 $\pm$ 0.0055 | 0.2403 $\pm$ 0.0064 | 0.4575 $\pm$ 0.0006 | 0.2762 $\pm$ 0.0050 |
| GRU4Rec | 0.2828 $\pm$ 0.0031 | 0.1836 $\pm$ 0.0019 | 0.4181 $\pm$ 0.0036 | 0.2275 $\pm$ 0.0021 |

表 2 给出 MovieLens-1M 派生数据的相同指标和统计口径。

| 模型      | HR@5                | NDCG@5              | HR@10               | NDCG@10             |
|---------|---------------------|---------------------|---------------------|---------------------|
| RoTE    | 0.5872 $\pm$ 0.0051 | 0.4230 $\pm$ 0.0036 | 0.7379 $\pm$ 0.0043 | 0.4719 $\pm$ 0.0039 |
| SASRec  | 0.5946 $\pm$ 0.0073 | 0.4315 $\pm$ 0.0077 | 0.7362 $\pm$ 0.0076 | 0.4774 $\pm$ 0.0082 |
| GRU4Rec | 0.4736 $\pm$ 0.0026 | 0.3209 $\pm$ 0.0022 | 0.6378 $\pm$ 0.0089 | 0.3740 $\pm$ 0.0041 |

在 Grocery 中，RoTE 相对 SASRec 的 NDCG@10 高 0.0024，约为 SASRec 均值的 0.9%；HR@5 和 NDCG@5 也略高，但 HR@10 反而低 0.0024。由此不能笼统地说 RoTE 全面优于位置编码。两者 NDCG@10 的差值小于各自三次运行的标准差，样本只有三个种子，尚不足以支持稳定优势或统计显著性判断。相较 GRU4Rec，两种注意力方法四项指标都更高，说明在该训练协议下注意力建模比此处的小型循环基线更适合，但差距不能单独归因于时间信息。

在 MovieLens 中，SASRec 的 NDCG@10 比 RoTE 高 0.0054，HR@5 和 NDCG@5 也较高；RoTE 的 HR@10 则高 0.0016。两个模型的差异随评价截断位置而变。GRU4Rec 的 NDCG@10 为 0.3740，明显低于其余两种方法。RoTE 和 SASRec 的三种子波动幅度与两者均值差相近，应将其理解为当前预算下的相近表现，而非宣称某模型在 MovieLens 上已被证明胜出。报告保留了每个种子的分数与原始日志，以便复核这些微小差异。

模型容量方面，Grocery 上 RoTE、SASRec、GRU4Rec 分别有 140848、141184、141328 个可训练参数；MovieLens 上分别为 51440、51776、51920。RoTE 相对 SASRec 少 336 个参数，主要对应移除的位置嵌入；主要容量仍来自物品嵌入。Grocery 的 RoTE 最佳轮次分别为 8、8、7，其余五组模型与数据组合的三个种子最佳轮次全部落在第 8 轮。这说明多数训练在预设上限处仍可继续改进，8 轮预算限制了充分收敛的判断。每次耗时和轮次见 results/summary.csv，但 MovieLens 并行运行及线程数不同使耗时无法作为受控的效率对照。

两个数据集的密度差异可能影响时间信号的作用：Grocery 训练阶段平均每名用户约 8.3 条交互，MovieLens 则约 93.2 条，而模型只取最近 20 条。MovieLens 的截断更频繁，Grocery 的短序列也使一年、月、日的间隔结构不同。这只是解释结果差异的可能因素，本实验没有做时间间隔分组或固定骨干的消融，不能证明因果关系。此外，Grocery 原始候选重复、重新采样负样本与 26 件冷启动留出物品都会影响绝对分数；所以跨数据集比较数值大小没有意义。

## 6 局限性与结论

本实验的时间编码遵循论文给出的年、月、日旋转与加权融合，但模型主体按 ReChorus 接口实现，与作者原始代码在数据、训练预算和可能的细节上存在差异。两个数据集的候选集均为 99 个采样负物品，评价范围小于全物品排序；Grocery 还含有冷启动留出物品。因此，结论只适用于本课程设定，不能推广为 RoTE 在所有推荐场景的优势或劣势。CPU 资源限制使嵌入维度与轮数低于常见大规模研究设定；若进一步研究，应控制相同骨干网络作去除时间旋转的消融，并增加时间间隔分层分析和更充分的超参数搜索。

综上，RoTE 的 UTC 累计年月日旋转已在 ReChorus CPU 流程中跑通，并在两套数据、两个同类基线上完成统一候选集比较。在 Grocery 的 NDCG@10 上它略高于 SASRec，在 MovieLens 的该指标上略低于 SASRec；两者差距都很小。课程复现证明了方法可以接入并复现可观察的时间敏感性，但没有提供“在这两套数据上稳定优于 SASRec”的证据。后续应延长训练、增加种子，并用同一注意力骨干进行去除时间旋转的对照，才能更清楚地区分时间编码本身的贡献。

## 7 小组分工与复现材料

史熠鑫负责 RoTE 模型接入、训练配置和三模型实验运行；王宸负责日期与数据处理测试、候选集审计、结果核对和报告检查。两人共同核实实验协议与结论。项目代码仓库地址为

<https://github.com/Tonybush811/rechorus-rote-cpu-coursework>，其中提供固定依赖、数据准备方法、README 和实验日志。

## 参考文献

- [1] Zhang H, Xiao L, Cai G, et al. RoTE: Coarse-to-Fine Multi-Level Rotary Time Embedding for Sequential Recommendation. SIGIR, 2026. DOI: 10.1145/3805712.3809892.
- [2] 王晨阳, 任一, 马为之, 张敏, 刘奕群, 马少平. ReChorus: 综合高效易扩展的轻量级推荐算法框架. 软件学报, 2022, 33(4): 1430-1438. DOI: 10.13328/j.cnki.jos.006473.
- [3] Kang W C, McAuley J. Self-Attentive Sequential Recommendation. ICDM, 2018.
- [4] Hidasi B, et al. Session-based Recommendations with Recurrent Neural Networks. ICLR, 2016.
- [5] GroupLens. MovieLens 1M Dataset. <https://grouplens.org/datasets/movielens/1m/>.